

Continuăm cu partea care îți va da rezultate reale pentru secțiunea **Evaluation**: un mic benchmark + error injection.

Vom adăuga:

```
src/  
  core.ts  
  examples.ts  
  benchmark.ts
```



1. Creează `src/benchmark.ts`

⟨⟩ TypeScript



```
import { task, seq, verify, compileToASL } from "./core";  
  
type A = { readonly tag: "A" };  
type B = { readonly tag: "B" };  
type C = { readonly tag: "C" };  
type D = { readonly tag: "D" };  
type E = { readonly tag: "E" };  
  
function measure<T>(f: () => T): { result: T; ms: number } {  
  const start = performance.now();  
  const result = f();  
  const end = performance.now();  
  return { result, ms: end - start };  
}  
  
function report(name: string, workflow: any) {  
  const v = measure(() => verify(workflow));  
  const errors = v.result;  
  
  let aslSize = 0;  
  try {  
    const asl = compileToASL(workflow);  
    aslSize = JSON.stringify(asl, null, 2).split("\n").length;  
  } catch {  
    aslSize = 0;  
  }  
  
  return {  
    name,  
    errors: errors.length,  
    contract: errors.filter(e => e.kind === "ContractError").length,  
    retry: errors.filter(e => e.kind === "RetrySafetyError").length,  
    compensation: errors.filter(e => e.kind === "CompensationError").length,  
    verificationMs: Number(v.ms.toFixed(3)),  
    generatedAslLoc: aslSize,  
  };  
};
```

```

}

/* Case Study 1: Order Processing */

const createOrder = task<A, B>("CreateOrder", "CustomerRequest", "OrderCreated");

const releaseInventory = task<C, B>(
  "ReleaseInventory",
  "OrderReserved",
  "OrderCreated"
);

const reserveInventory = task<B, C>(
  "ReserveInventory",
  "OrderCreated",
  "OrderReserved"
)
.persistentTask()
.compensate(releaseInventory);

const chargeCard = task<C, D>(
  "ChargeCard",
  "OrderReserved",
  "OrderPaid"
).nonIdempotent();

const shipOrder = task<D, E>("ShipOrder", "OrderPaid", "OrderShipped");

const orderGood = seq(
  createOrder,
  reserveInventory,
  chargeCard,
  shipOrder
);

const orderUnsafeRetry = seq(
  createOrder,
  reserveInventory,
  chargeCard.retry(),
  shipOrder
);

/* Case Study 2: Travel Booking */

const reserveFlight = task<A, B>(
  "ReserveFlight",
  "TravelRequest",
  "FlightReserved"
).persistentTask();

const cancelFlight = task<B, A>(
  "CancelFlight",

```

```
    "FlightReserved",
    "TravelRequest"
);

const reserveHotel = task<B, C>(
    "ReserveHotel",
    "FlightReserved",
    "HotelReserved"
).persistentTask();

const cancelHotel = task<C, B>(
    "CancelHotel",
    "HotelReserved",
    "FlightReserved"
);

const payTravel = task<C, D>(
    "ChargeCard",
    "HotelReserved",
    "TravelPaid"
).nonIdempotent();

const travelGood = seq(
    reserveFlight.compensate(cancelFlight),
    reserveHotel.compensate(cancelHotel),
    payTravel
);

const travelMissingCompensation = seq(
    reserveFlight,
    reserveHotel.compensate(cancelHotel),
    payTravel
);

/* Case Study 3: Data Pipeline */

const readData = task<A, B>("ReadData", "InputDataset", "RawData").idempotentTask();

const transformData = task<B, C>(
    "TransformData",
    "RawData",
    "TransformedData"
).idempotentTask();

const validateData = task<C, D>(
    "ValidateData",
    "TransformedData",
    "ValidatedData"
).idempotentTask();

const storeResults = task<D, E>(
    "StoreResults",
```

```

    "ValidatedData",
    "StoredResults"
  ).persistentTask();

const rollbackStore = task<E, D>(
  "RollbackStore",
  "StoredResults",
  "ValidatedData"
);

const dataGood = seq(
  readData.retry(),
  transformData.retry(),
  validateData.retry(),
  storeResults.compensate(rollbackStore)
);

const dataMissingCompensation = seq(
  readData.retry(),
  transformData.retry(),
  validateData.retry(),
  storeResults
);

/* Run benchmark */

const workflows = [
  ["OrderGood", orderGood],
  ["OrderUnsafeRetry", orderUnsafeRetry],
  ["TravelGood", travelGood],
  ["TravelMissingCompensation", travelMissingCompensation],
  ["DataGood", dataGood],
  ["DataMissingCompensation", dataMissingCompensation],
] as const;

const rows = workflows.map(([name, wf]) => report(name, wf));

console.table(rows);

const totals = {
  workflows: rows.length,
  totalErrors: rows.reduce((s, r) => s + r.errors, 0),
  contractErrors: rows.reduce((s, r) => s + r.contract, 0),
  retryErrors: rows.reduce((s, r) => s + r.retry, 0),
  compensationErrors: rows.reduce((s, r) => s + r.compensation, 0),
  avgVerificationMs: Number(
    (
      rows.reduce((s, r) => s + r.verificationMs, 0) / rows.length
    ).toFixed(3)
  ),
};

```

```
console.log("\nSummary:");
console.table([totals]);
```

2. Rulează benchmark-ul

</> Bash



```
npx ts-node src/benchmark.ts
```

Vei obține un tabel de forma:

```
OrderGood          0 errors
OrderUnsafeRetry    1 retry error
TravelGood          0 errors
TravelMissingCompensation 1 compensation error
DataGood           0 errors
DataMissingCompensation 1 compensation error
```



Asta deja îți oferă rezultate reale pentru lucrare.

3. Cum transformi în tabel pentru paper

După ce rulezi, înlocuiești tabelele inventate din Evaluation cu ceva de forma:

</> LaTeX



```
\begin{table}[t]
\centering
\caption{Verification results on benchmark workflows.}
\label{tab:verification-results}
\begin{tabular}{lcccc}
\hline
Workflow & Errors & Contract & Retry & Compensation \\
\hline
OrderGood & 0 & 0 & 0 & 0 \\
OrderUnsafeRetry & 1 & 0 & 1 & 0 \\
TravelGood & 0 & 0 & 0 & 0 \\
TravelMissingCompensation & 1 & 0 & 0 & 1 \\
DataGood & 0 & 0 & 0 & 0 \\
DataMissingCompensation & 1 & 0 & 0 & 1 \\
\hline
\end{tabular}
\end{table}
```

4. Următorul pas important

Acum trebuie să adăugăm și **contract/typestate error examples**. Problema este că, în versiunea actuală, `seq(...)` prinde unele incompatibilități la TypeScript compile-time,

deci nu le putem pune direct în benchmark fără să stricăm compilarea.

Soluția pentru paper este bună:

⌕ TypeScript



```
unsafeSeq(...)
```

adică o funcție care permite construirea intenționată de workflow-uri greșite pentru error injection.

În pasul următor adăugăm `unsafeSeq`, iar benchmark-ul va produce și contract/typestate violations reale.